

Memory-Efficient Test-Time Scaling and Latent Reasoning in Multilingual Language Models

The Tokenization Tax on Thinking: Latent vs. Explicit Test-Time Compute for Indic Reasoning Under Memory Constraints

Item	Details
Team Number	PW26_BJD_18
Team	P Saanvi – PES1UG24AM905 Rohini Vishu – PES1UG24AM922 Aditya Vats – PES1UGCS032 Aiman Nawab – PES1UG24CS038
Guide	Dr. Bhaskarjyoti Das

1. Summary

Test-time scaling (TTS) makes small language models reason better by sampling many solution paths, but every path needs its own key-value (KV) cache, so memory grows with the number of paths multiplied by trace length. Indic scripts need roughly 2-4x more tokens than English for the same content, so Indic reasoning reaches the memory limit of a single GPU far sooner. Latent reasoning (COCONUT-style continuous thoughts) skips token generation during thinking, so its thinking cost does not depend on the tokenizer. This project measures the tokenization tax on explicit TTS, tests how much quantization and probe-guided pruning can recover, and asks whether latent reasoning is a cheaper substitute for explicit search at matched memory, in English, Hindi and Telugu/Kannada.

2. Problem Statement and Gap

2.1 The problem

Small reasoning models are the realistic option for consumer and edge GPUs, which is where most Indian deployments will run. On these devices, memory rather than compute limits how much test-time search is possible. For a 0.6B model such as Qwen3-0.6B (about 112 KB of KV cache per token in fp16), 32 parallel 4k-token traces already need about 15 GB. The same setting with Telugu traces at 2.5x the length needs about 37 GB and no longer fits on a 24 GB card.

2.2 Why this is a gap

- **No measurement of the tax.** Work on KV-cache quantization (KIVI) and probe-based branch pruning (ReProbe) is almost entirely in English. To our knowledge, nobody has measured how tokenization inflates the memory cost of TTS across languages.
- **The English pivot is only studied through the vocabulary.** Logit-lens studies show multilingual models reason through an English-like space in their middle layers, with weaker convergence for low-resource languages. All of this evidence reads internal states as tokens. Nobody has asked what happens to the pivot in a model whose reasoning loop never touches the vocabulary.
- **Latent reasoning has not been trained multilingually.** Existing multilingual latent-reasoning work probes emergent behaviour in ordinary models. We know of no work that trains an architecturally latent model (COCONUT) on Indic languages, or that compares latent and explicit reasoning on an accuracy-versus-peak-memory axis.
- **Recent papers ask for exactly this.** Recent work calls for necessity-first evaluation of latent reasoning, and TTS surveys name memory as the main deployment barrier.

3. Research Questions

ID	Focus	Question
RQ1	The pivot	When the vocabulary projection is removed from the reasoning loop, does the English pivot survive or dissolve?
RQ2	The tax	How much does tokenization inflate the memory cost of explicit TTS in Hindi and Telugu, and how much can KV quantization and probe-guided pruning recover on a 24 GB GPU?
RQ3	The bridge	At matched peak memory, how does latent reasoning compare with explicit search across languages? How robust are continuous thoughts to quantization, and can a probe decide how many latent steps to take?

Every outcome is informative: the pivot surviving or dissolving, a large or small tax, and a latent advantage or none are all publishable findings.

4. Proposed Approach

The project has three pillars, each of which can be published on its own, so a failure in one does not sink the whole project.

- **Pillar B, memory-constrained explicit TTS (RQ2).** Sweep the number of samples K and KV precision (fp16, 8, 4, 2-bit) across languages using vLLM or llama.cpp with existing KV quantization support. Train a ReProbe-style probe to prune weak branches early. Report accuracy against peak memory.
- **Pillar A, English pivot in latent models (RQ1).** Train COCONUT-style latent models on Qwen3-0.6B-Base, with controls branched from the stage-0 chain-of-thought checkpoint. Analyse representations with probing and causal tracing.
- **Pillar C, latent vs. explicit at matched memory (RQ3).** Mostly inference on the checkpoints and harness from Pillars A and B. Includes quantization robustness of latent thoughts and probe-guided adaptive latent depth.

Models: Qwen3-0.6B-Base (primary), Gemma 3 270M (debugging only), Sarvam-1 tokenizer (tokenizer-only comparison, to separate "Indic languages are harder" from "Indic languages are tokenized worse").

5. Possible Datasets and Evaluation

Dataset / resource	Use	Notes
GSM8K-style augmented reasoning traces (about 40k problems)	Training data for chain-of-thought and latent runs	Translated into Hindi and Telugu with IndicTrans2; examples where translation changed a number or the final answer are filtered out
MGSM (250 problems per language)	Evaluation only, for Telugu and Bengali	Human-translated, so it is the cleanest multilingual test set available
Held-out translated GSM8K test split	Evaluation for languages MGSM does not cover	Same filtering rules as the training data
Tokenizer fertility corpus (parallel text in English, Hindi, Telugu)	Tokenization-tax measurement for Qwen3, Gemma 3 and Sarvam-1	CPU only; first result of the project
Fourth language (flexible)	Check generalisation	Bengali is convenient because MGSM covers it, but a second Dravidian language gives more typological distance

Primary metric: accuracy against peak GPU memory, per language. **Secondary metrics:** tokens per word (fertility), KV-cache bytes per solved problem, and probe accuracy for pruning and for latent-step selection.

6. Application and Impact

- **Who will use it:** teams deploying small reasoning models on consumer or edge GPUs, including startups and public-sector or education tools serving Hindi and Telugu speakers. Researchers in multilingual and efficient NLP are the second audience.
- **Where:** memory-limited and offline settings such as tutoring apps, regional-language assistants and on-device deployments.
- **Impact:** (1) a quantified tokenization tax that guides tokenizer and model choice for Indic languages, (2) practical guidance on whether to spend a fixed memory budget on explicit search or latent reasoning, (3) interpretability evidence on whether latent reasoning is English thinking in disguise, and (4) open-source code, translated reasoning data and an evaluation protocol.

7. Roadmap

Phase	Timeline	Work	Outcome
Phase 0 Go/no-go checks	Sem 5	Tokenizer fertility for Qwen3, Gemma 3 and Sarvam-1 on English, Hindi, Telugu Logit-lens pivot check on Qwen3-0.6B-Base Baseline MGSM accuracy per language Reproduce official COCONUT repo on GPT-2 (GSM8K) IndicTrans2 trial on a small sample	Go/no-go memo First tokenization-tax numbers Sequence-length budgets for training
Phase 1 Pillar B + data	Winter break – sem 6 mid	Build the TTS sweep harness over K and KV precision Train the pruning probe Translate and filter the training data Write up Pillar B results	Working code release (v0.1) Accuracy-vs-memory curves per language Translated dataset Workshop paper draft
Phase 2 Pillar A	Sem 6	Stage gate: English and Telugu, one seed If the signal holds, full grid: 4 languages, 3 seeds, plus two controls Probing and causal tracing	Latent checkpoints Pivot analysis (RQ1) Optional early workshop submission
Phase 3 Pillar C + writing	Summer break to mid Sem 7	Latent vs. explicit at matched memory Quantization robustness of latent thoughts Probe-guided adaptive latent depth Full paper writing and internal review	Matched-memory results (RQ3) Full paper submitted by mid Sem 7 Public benchmark and code

7.1 Phase 1 in detail:

Module	What it does	Acceptance check
tokenization-tax calculator	Computes tokens per word for each tokenizer and language, and projects KV-cache memory per trace length and K	Reproduces the 2-4x inflation estimate on real data; table for all three tokenizers
translation + filter pipeline	Runs IndicTrans2 over GSM8K-style traces and drops examples where numbers or final answers changed	Filtering report with the share of examples kept per language
baseline evaluator	Scores Qwen3-0.6B-Base on MGSM and the translated test split	Per-language baseline accuracy table
TTS sweep harness	Runs best-of-K search over K and KV precision (fp16, 8, 4, 2-bit), logging accuracy and peak GPU memory	Accuracy-vs-memory curves for English, Hindi and Telugu from one command
pruning probe	ReProbe-style probe on internal states that stops weak branches early	Memory saved at equal or better accuracy than unpruned best-of-K
repository	Configs, seeds, README and run scripts	A teammate can reproduce one headline figure from a clean checkout

8. Publication Plan

Submission	When	Content	Venue type
Workshop paper (optional early output)	submit in Sem 6 when a suitable deadline opens	Pillar B: tokenization tax and memory-constrained TTS across languages	Efficiency or multilingual workshop
Main paper	By mid Sem 7	Pillars A, B and C: pivot analysis, tokenization tax, and latent vs. explicit at matched memory	ACL or EMNLP family; latent-reasoning workshop as a fallback

9. Risks and Fallbacks

Risk	Fallback
Qwen3-0.6B shows no English pivot	Reframe Pillar A as whether latent training creates a shared cross-lingual space; lean on Pillars B and C
Translated data is too noisy	Stricter answer-consistency filtering, drop to three languages, or check AI4Bharat releases for native data
KV quantization gains are unremarkable (crowded area; KIVI is direct prior art)	The contribution is the multilingual tokenization-tax measurement, not a new quantizer
0.6B model too weak for TTS to help in Telugu	Add an easier or synthetic task tier where base accuracy is non-trivial
Timeline pressure before mid Sem 7	Pillar C is mostly inference, so it can run in parallel with Pillar A write-up; fall back to a B+C paper
Rented GPU sessions are interrupted	Checkpoint every 30-60 minutes; all runs are single-GPU and resume easily

10. Resources

RTX 4090 (24 GB). Estimated total is about 450-700 GPU-hours including a 30% buffer, split roughly as Phase 0 at 30-40 hours, Pillar B at 100-150, data translation at 15-25, Pillar A at 150-250, and Pillar C at 50-100. All figures are plus or minus 50% until Phase 0 measures real throughput. Seeds will run in parallel on separate cards.

11. Key References

Latent Reasoning

- Hao et al. (2024). *Training LLMs to Reason in a Continuous Latent Space (COCONUT)*. arXiv:2412.06769
- Geiping et al. (2025). *Scaling Test-Time Compute with Latent Reasoning (Huginn)*. arXiv:2502.05171

Multilingual Latent Language and the English Pivot

- Wendler et al. (2024). *Do Llamas Work in English? On the Latent Language of Multilingual Transformers*. arXiv:2402.10588
- Liu et al. (2026). *Large Reasoning Models Are (Not Yet) Multilingual Latent Reasoners*. arXiv:2601.02996

Test-Time Scaling and Memory

- Muennighoff et al. (2025). *s1: Simple Test-Time Scaling*. arXiv:2501.19393
- FastTTS (2026). *Accelerating Test-Time Scaling for Edge LLM Reasoning*. ASPLOS 2026. arXiv:2509.00195

KV-Cache Quantization and Lightweight Verification

- Liu et al. (2024). *KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache*. arXiv:2402.02750
- Ni et al. (2026). *ReProbe: Efficient Test-Time Scaling of Multi-Step Reasoning by Probing Internal States*. ACL 2026. arXiv:2511.06209

Data, Translation and Evaluation

- Gala et al. (2023). *IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for all 22 Scheduled Indian Languages*. arXiv:2305.16307
- Shi et al. (2022). *Language Models are Multilingual Chain-of-Thought Reasoners (MGSM)*. arXiv:2210.03057
- Cobbe et al. (2021). *Training Verifiers to Solve Math Word Problems (GSM8K)*. arXiv:2110.14168